Katholieke
Universiteit
Leuven

**Department of
Computer Science**

A Reverse-Hoops-Playing Robot Arm

Robotics (B-KUL-H02A4A)

Enmin Lin (r0927480), Kaixi Yao (r0926887)

Academic year 2025–2026

Reverse-Hoops-Playing Robot Arm

System Overview and Experimental Setup

The task is simple to state: the ball is thrown at the robot, and the UR10 must move the hoop so that the ball passes through it [1]. The hard part is that the ball measurements are noisy, the robot has joint limits, and the ball only has 30 mm of radial clearance inside the hoop. The system is split into three questions. Where will the ball go? Which future point should the robot try to catch? Can the UR10 move the hoop there without breaking its limits?

The main claim of this report is that the second question matters most. The Kalman filter and the NLP controller are kept fixed in the benchmark. Only the interception-point rule changes. This makes the comparison easy to read: a better prediction model alone is not enough, and a point that looks good geometrically may still be impossible for the robot.

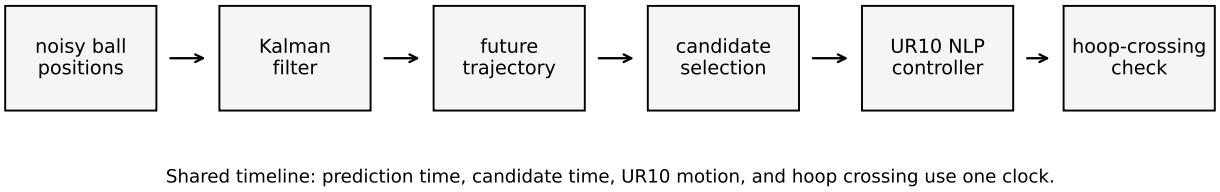

Figure 1: System pipeline used in the benchmark. The same ball prediction, candidate pool, NLP controller, and hoop-crossing metric support all strategy comparisons.

Table 1: Constants and modelling choices used by the estimator, controller, and success metric.

Item	Value	Quantitative role
Ball and hoop radii	0.12, 0.15 m	$r_h - r_b = 30\,\text{mm}$ radial tolerance
Measurement noise	1 mm Gaussian position noise	$R = \sigma^2 I_3$ in the Kalman update
Robot base	Fixed	Removes mobile-base planning from the report
Acceleration limit	$2\,\text{rad/s}^2$	$\lvert\ddot{q}_{k,j}\rvert \le 2$ for every knot and joint
Hoop orientation	Top side never faces ground	TCP z-axis satisfies $R_{zz}(q_k) \ge 0$
Table clearance	Hoop center remains above table	$z_{\text{tcp}}(q_k) \ge h_{\text{table}} + m_{\text{tcp}}$, hard path constraint
Self-collision proxy	Link-sphere clearance check	Positive sphere clearance means no proxy overlap after planning

The simulator gives the current robot state and noisy ball positions. Only the ball state is estimated. After a candidate point is chosen, the controller plans one joint trajectory and then executes it open-loop; no MPC is used during the simulation. This is why the benchmark reports two different errors. Terminal TCP error says how well the NLP reaches the predicted target. Radial crossing error says whether the true ball actually passes through the hoop. The simulator includes bounce physics, but every reported catch happens before the first bounce. A post-bounce catch would need a different prediction model.

Task 1: Trajectory Estimation under Position Noise

The ball state uses position and velocity,

$$x_t = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^\top. \tag{1}$$

The process model keeps horizontal velocity constant and adds known gravity in z,

$$x_{t+\Delta t} = Fx_t + Bg + w_t, \qquad y_t = Hx_t + v_t, \qquad v_t \sim \mathcal{N}(0, \sigma^2 I_3). \tag{2}$$

Here F advances the constant-velocity state, Bg adds gravity, H selects the position entries, and y_t is the measured ball position. Between bounces, this is a linear model. The project also gives Gaussian position noise with known standard deviation.

Filter choice. A Kalman filter fits these assumptions directly. The belief is Gaussian, the dynamics are linear, and the measurement model is linear with Gaussian noise. Each step first predicts the mean and covariance, then corrects them with the measurement innovation. Under this model, the Kalman update is the Bayes-optimal MMSE estimate and also gives an analytic covariance [2]. The covariance is useful because it shows whether prediction uncertainty is small compared with the 30 mm hoop clearance. An EKF would reduce to the same update here, and a particle filter would add sampling noise without adding useful modelling power.

The filter starts from the first noisy ball position. Its initial velocity is set to zero, but with a large velocity covariance so that later measurements can quickly correct it. The robot state is available from the simulator, so the estimator only tracks the

ball. Across 50 seeds at the project noise level, raw measurement RMSE was 1.74 mm, filtered RMSE was 0.90 mm, and 0.5 s future-prediction RMSE was 3.51 mm. The 95th percentile future-prediction RMSE was 6.35 mm, still far below the 30 mm hoop clearance.

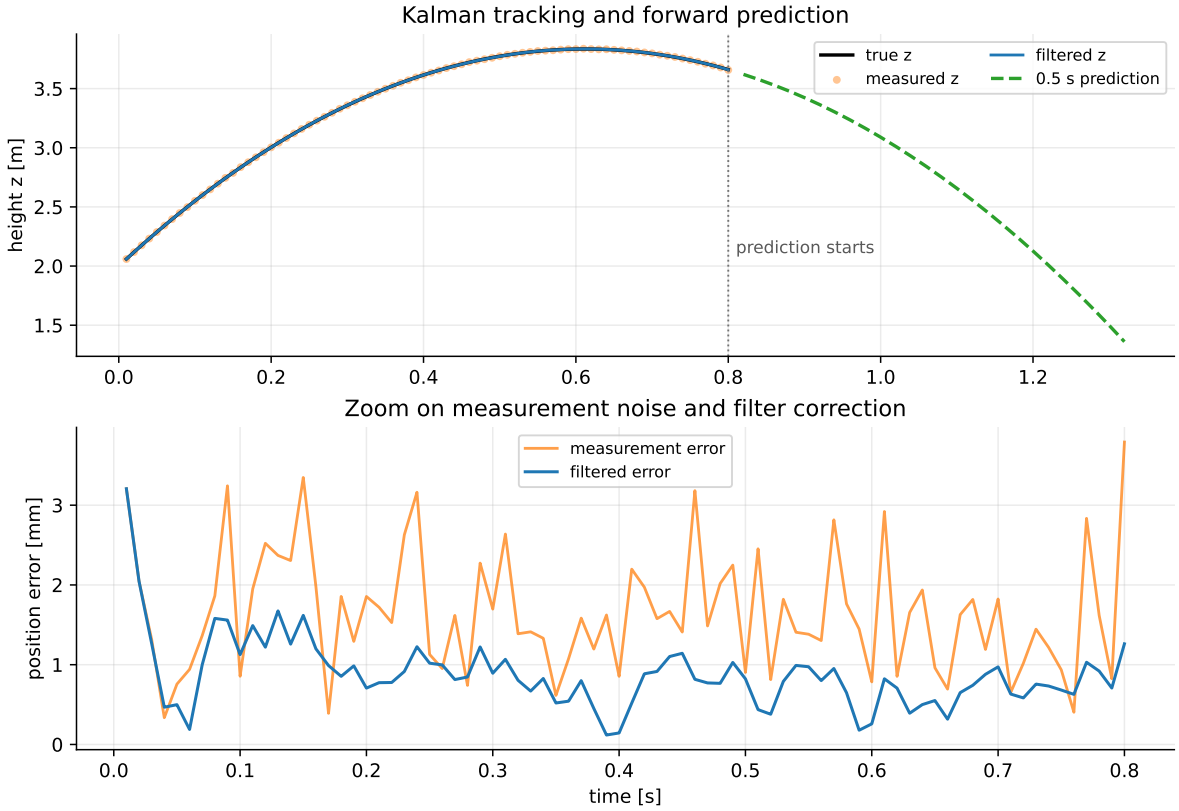

Figure 2: Kalman-filter prediction for one seed. The lower panel shows that filtering removes most of the 1 mm measurement jitter, and the 50-seed validation keeps the 0.5 s prediction error below 6.35 mm at the 95th percentile.

A noise sweep checks how much this result depends on the exact noise level. When measurement noise increases from 1 to 10 mm, mean 0.5 s prediction RMSE increases from 3.67 to 10.34 mm, and the largest observed RMSE stays at 20.01 mm. The sweep uses a different seed set, so the small difference between 3.51 mm and 3.67 mm is just sampling variation. Even at higher noise, the prediction error remains below the 30 mm radial clearance. Estimation is not the main bottleneck in this setup.

Task 2: Geometric Candidate Selection Baselines

Task 2 only asks for a simple interception point. The baseline therefore does not solve a robot motion problem yet. It samples the predicted ball trajectory and rejects points that are too soon, too low, or too far from the current hoop center. The standalone Task 2 validation uses a 0.85 m workspace radius. We use this coarse workspace-radius filter as a simple reachability proxy, and the later Task 3 tests confirm that this proxy is conservative. The Task 4 benchmark uses a wider 1.15 m radius so that all strategies start from the same larger candidate pool. With that wider pool, the simple rule tries earlier points, so its 50-seed mean attempt time is 1.295 s, earlier than the standalone 1.36 s pick. For the validation seed, the rule selected $[1.224, -0.119, 1.074]$ m.

This rule answers only one question: where does the ball pass through a reasonable region of space? It does not check whether the UR10 can move the hoop there with the right pose and acceleration. Across 50 seeds, the simple geometric rule attempted an early catch at 1.295 ± 0.012 s, but it produced 0 successful catches under the full NLP and hoop-crossing check.

A second baseline removes prediction completely. It uses a damped Jacobian pseudo-inverse Cartesian velocity controller,

$$\dot{q} = J(q)^\top \left(J(q)J(q)^\top + \lambda^2 I \right)^{-1} K(p_{\text{ball,meas}} - p_{\text{tcp}}), \tag{3}$$

with proportional gain K, damping λ, and velocity and acceleration saturation. This reactive controller also gives 0% success. Its closest TCP-to-ball distance was 0.176 ± 0.058 m, and even the best seed stayed 59.4 mm away, about twice the 30 mm radial tolerance. It can chase the ball, but it cannot arrive at the right place at the right time. Prediction is necessary, and a predicted point by itself is still not enough.

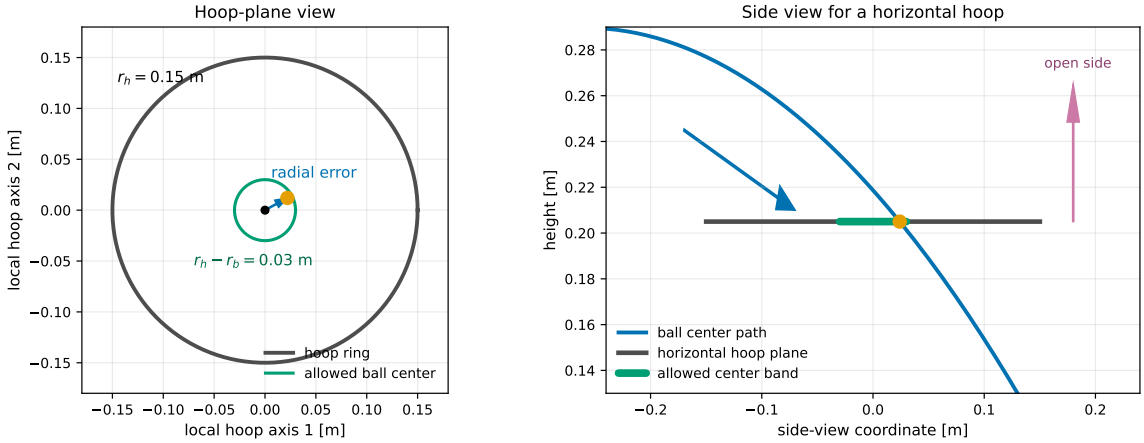

Figure 3: Success metric used in Task 3 and Task 4. The ball center must cross the hoop plane and lie within the $r_h - r_b = 30\,\text{mm}$ radial band; otherwise the ball touches the ring or misses the opening.

Task 3: Constrained Trajectory Optimization for the UR10

The controller plans the robot motion with a finite-horizon nonlinear program. The decision variables are joint position, velocity, and acceleration,

$$\{q_k, \dot{q}_k\}_{k=0}^{N}, \quad \{\ddot{q}_k\}_{k=0}^{N-1}. \tag{4}$$

The plan must obey the second-order joint dynamics

$$q_{k+1} = q_k + \dot{q}_k \Delta t + \tfrac{1}{2}\ddot{q}_k \Delta t^2, \qquad \dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t. \tag{5}$$

The optimization problem is

$$\begin{aligned} \min_{q,\dot{q},\ddot{q}} \; & w_p \|p_{\text{tcp}}(q_N) - p^\star\|_2^2 + \sum_{k=0}^{N-1} w_u \|\ddot{q}_k\|_2^2 + \sum_{k=1}^{N} w_v \|\dot{q}_k\|_2^2 \\ & + w_o (1 - R_{zz}(q_N))^2 + w_n \big(1 - (n_{\text{hoop}}(q_N)^\top \hat{v}_{\text{ball}})^2\big) \\ \text{s.t. } & q_0 = q_{\text{init}}, \quad \dot{q}_0 = \dot{q}_{\text{init}}, \\ & q_{\min} \le q_k \le q_{\max}, \quad -\dot{q}_{\max} \le \dot{q}_k \le \dot{q}_{\max}, \quad -\ddot{q}_{\max} \le \ddot{q}_k \le \ddot{q}_{\max}, \\ & z_{\text{tcp}}(q_k) \ge h_{\text{table}} + m_{\text{tcp}}, \quad R_{zz}(q_k) \ge 0, \qquad \forall k. \end{aligned} \tag{6}$$

Equation 5 prevents the optimizer from jumping between unrelated joint states. In Eq. 6, $p^\star$ is the predicted ball position at the catch time, p_{tcp} is the hoop-center position, n_{hoop} is the hoop normal, and $\hat{v}_{\text{ball}}$ is the unit ball velocity at the catch point. The first cost term moves the hoop center to the ball. The next two terms keep acceleration and velocity small. The last two terms keep the hoop upright and align the hoop with the ball direction. The squared dot product treats the two normal directions symmetrically, while the hard $R_{zz} \ge 0$ path constraint keeps the upright branch. The inequalities enforce joint position, velocity, acceleration, table-clearance, and top-side orientation limits.

The validation uses $N = 28$, $\Delta t = 0.05\,\text{s}$, $w_p = 1500$, $w_u = 2 \times 10^{-2}$, $w_v = 10^{-3}$, $w_o = 10$, and $w_n = 5$. This gives a $1.4\,\text{s}$ validation horizon. In the benchmark, the horizon is recomputed for each candidate catch time. In the URDF, the fixed tool-to-TCP transform maps the physical ring normal to the TCP z-axis. The controller therefore uses TCP z as n_{hoop}. The scalar $R_{zz} = e_z^\top R_{\text{tcp}}(q) e_z$ is the world-z component of that axis. The constraint $R_{zz}(q_k) \ge 0$ keeps the top side from pointing below the horizontal plane. Pinocchio supplies URDF forward kinematics [5]. CasADi builds the symbolic NLP [4], and IPOPT solves it [3].

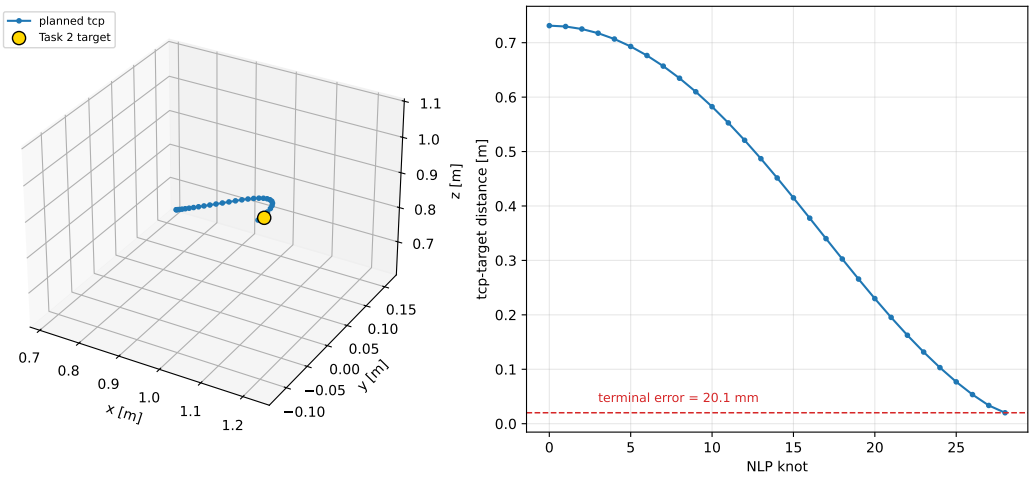

Figure 4: Task 3 finite-horizon NLP result. The hoop center reaches 20.1 mm terminal TCP error while the path constraints keep the motion executable.

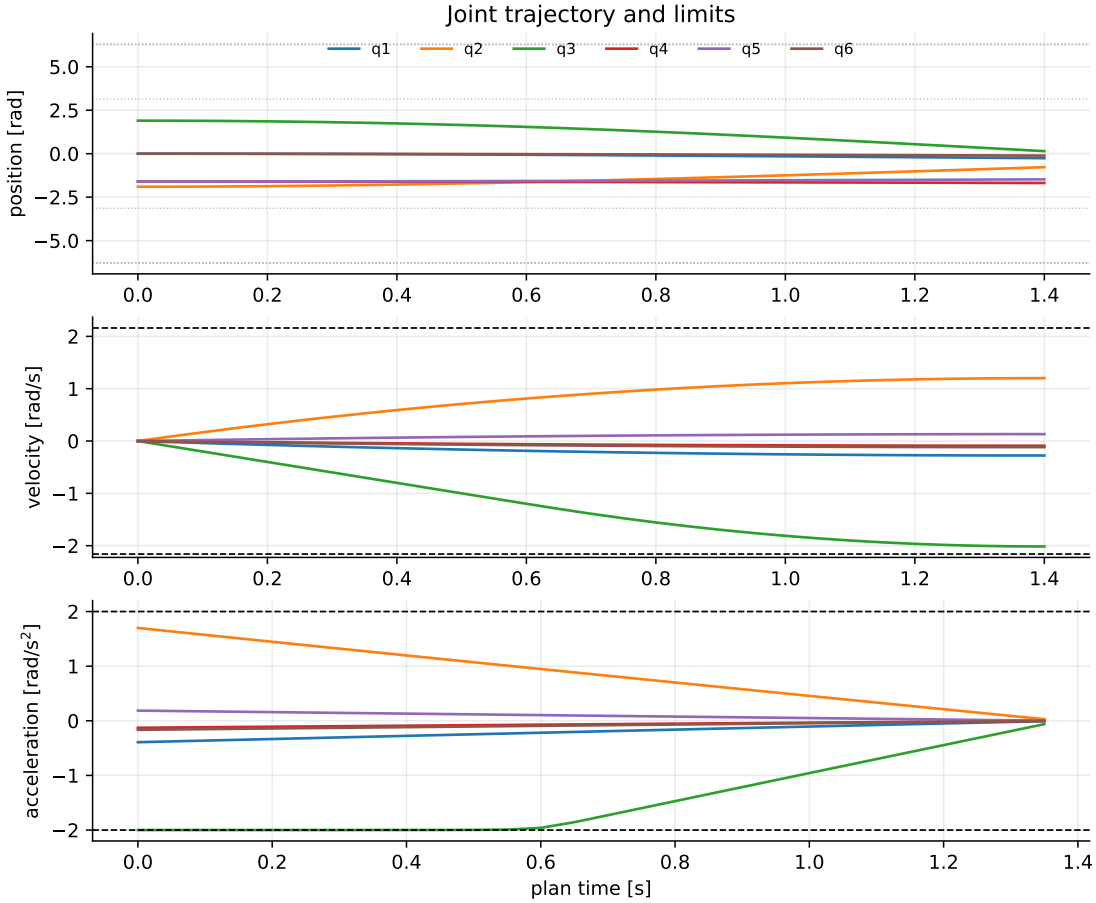

Figure 5: Joint profiles for the same plan. All accelerations approach but do not exceed $\pm 2\,\mathrm{rad/s^2}$, and joint 2 reaches the highest velocity utilization at 64% of its limit.

Figure 4 shows what the NLP does in space: the hoop center moves from a large initial offset to a 20.1 mm terminal error at the selected catch point. Figure 5 shows why the motion is difficult. The robot is not mainly limited by velocity; joint 2 uses only 64% of its velocity bound. The tight limit is acceleration, because the acceleration profile touches the $\pm 2\,\mathrm{rad/s^2}$ bound.

For the Task 2 target, the NLP reduced the hoop-target distance from 0.731 m to 0.020 m. It solved in 0.071 s and 15 IPOPT iterations on the validation run. The maximum acceleration was $1.9996\,\mathrm{rad/s^2}$, the maximum velocity ratio was 0.640, and the minimum R_{zz} value was 0.732. The minimum TCP-table clearance was 0.622 m, the minimum frame-table clearance was 0.115 m, and the minimum self-collision clearance proxy was 0.163 m. Table clearance and hoop orientation are hard NLP path constraints. Self-collision is checked after planning with a conservative link-sphere proxy, not with exact mesh distances.

This result should be read as a controller validation. Task 3 shows that the UR10 can move the hoop center close to the predicted target. Final catch success still depends on where the true ball crosses the hoop plane, so the full catch check is evaluated in Task 4.

Exact mesh self-collision would add many non-smooth distance constraints. For this report, all successful plans are required to keep positive clearance under the sphere proxy. A stricter version could add capsule or sphere signed-distance constraints

directly to Eq. 6.

The w_o ablation checks whether the upright-orientation soft cost is doing the main work. It is not. Across $w_o \in \{0, 1, 10, 50\}$, time-first benchmark success stayed at 98%. The hard $R_{zz} \geq 0$ path constraint and the normal-alignment term set the final geometry. The terminal R_{zz} cost mainly regularizes the pose.

Task 4: Feasibility-Aware Candidate Selection

Task 4 asks for a smarter interception point with higher success rate and shorter interception time. The submitted strategy is the time-first feasible selector. It chooses the earliest candidate that passes the predicted-candidate, NLP-feasibility, and safety checks. The selector is computed from predicted candidates, NLP feasibility, and safety metrics. The true-ball hoop-crossing check is used as the benchmark success metric after selection. The balanced and late-feasible selectors are kept as comparison cases, because they show what is gained by waiting longer.

The simple geometric rule tries an earlier catch, but it has 0% success. Its time is therefore only an attempted time, not a successful catch time. For a fair comparison, all policies are evaluated on the same candidate pool. The selector samples the predicted trajectory and solves the Task 3 NLP for each shortlisted candidate. A benchmark attempt is counted as successful only if the NLP converges, the true ball crosses the hoop plane inside the tolerance, and the safety penalty is zero. All 396 evaluated candidate NLPs converged, but only 164 candidate attempts passed the hoop-crossing test. On average, each seed had 7.9 evaluated candidates and 3.3 benchmark-successful catches. About 59% of geometrically plausible candidates were rejected after the full check. This is the main bottleneck: the robot can often solve the motion plan, but many candidate times do not put the hoop in the right crossing geometry. Compared with the simple geometric attempt at 1.295 s, the time-first feasible selector catches 92 ms later. That number is catch-time delay, not computation time. The delay is about two control ticks because the NLP uses $\Delta t = 50$ ms, and it raises success from 0% to 98%.

The balanced selector ranks feasible candidates by catch time, terminal TCP error, acceleration use, velocity use, and safety penalty, with weights $15.0, 100.0, 0.20, 0.20, 50.0$. The time-first and late-feasible policies use the same candidate pool but select the earliest or latest successful candidate. Figure 6 shows the difference visually: red marks the simple geometric pick, and yellow marks the feasible catch that the UR10 can actually make.

Two extra checks point to the same conclusion. The bottleneck is geometric feasibility, not estimator quality. In a 1–10 mm noise sweep over 20 downstream seeds, time-first success stayed at 95% for every tested noise level. With 20 seeds, one failure changes the rate by 5%, so this matches the 50-seed 98% result. A covariance-rejection ablation also selected the same catches as time-first feasible across all 50 seeds when it removed candidates above a conservative 0.18 m forward-covariance threshold. The covariance is useful for diagnosis, but it is not the deciding signal in this open-loop benchmark.

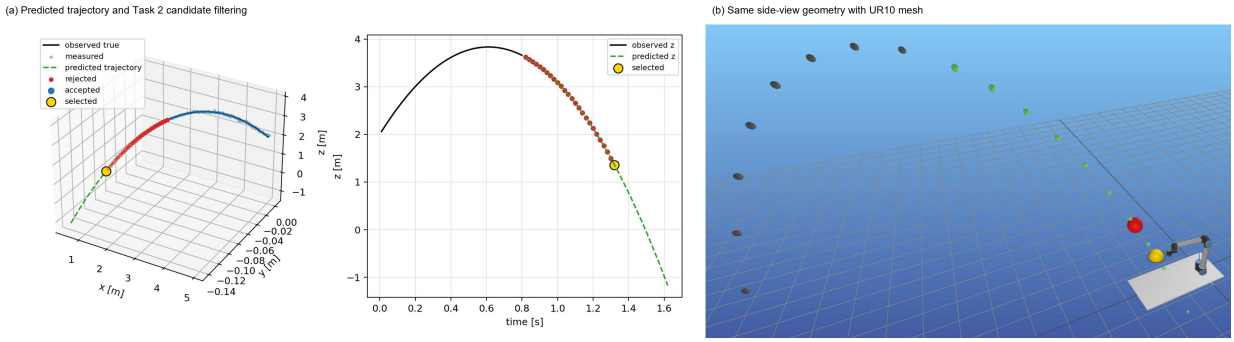

Figure 6: Task 2 and Task 4 candidate geometry in matching views. The left panels show predicted-trajectory filtering for the candidate pool; the right panel renders the same scene in Meshcat. Colors: grey = observed past trajectory, green = predicted future trajectory, red = simple geometric candidate, yellow = selected feasible catch.

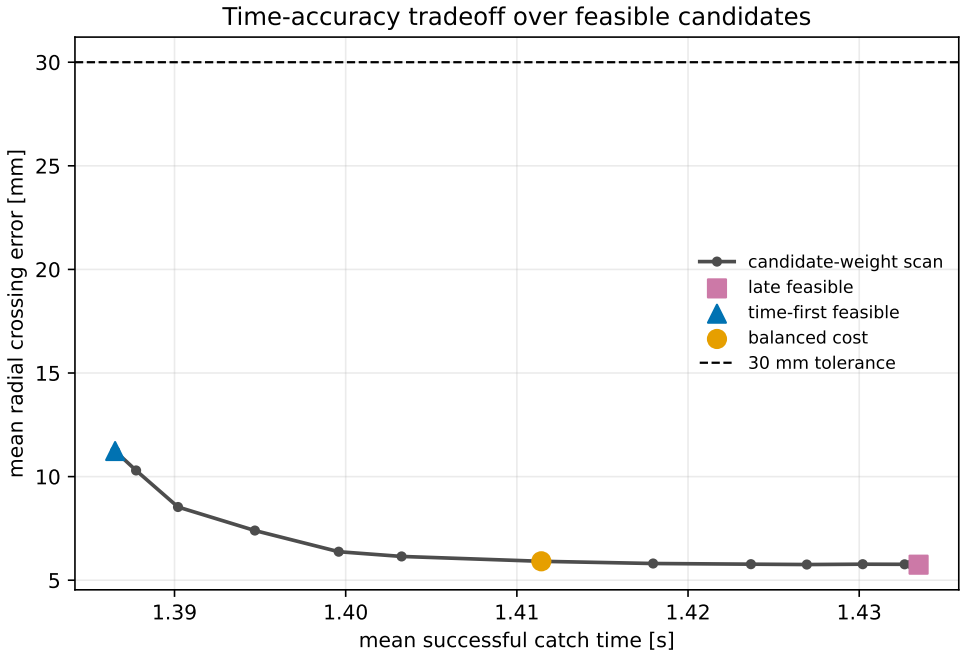

Figure 7: Empirical candidate-weight scan on the time-accuracy plane. Time-first gives the early-time end, late feasible gives the accurate but slower end, and the elbow near balanced cost buys roughly 5 mm of radial accuracy for about 25 ms.

Table 2: Final 50-seed benchmark. Time is catch time from ball release. TCP is terminal hoop-center error to the predicted target. Radial is true hoop-plane crossing error. Max $\ddot{q}$ is the largest absolute joint acceleration in rad/s^2, with hard bound 2. The asterisked simple time is attempted, not successful.

Strategy	Success	Time [s]	TCP [mm]	Radial [mm]	Max $\ddot{q}$	IPOPT
Reactive no prediction	0%	–	–	–	–	–
Simple geometric	0%	1.295*	–	–	–	–
Late feasible	98%	1.433 ± 0.009	1.87	5.75	1.65	10.4 it.
Time-first feasible	98%	1.387 ± 0.018	13.18	11.24	2.00	13.5 it.
Balanced cost	98%	1.411 ± 0.015	3.22	5.91	1.81	12.3 it.

The time-first feasible policy is the clearest answer to Task 4. It raises full-system success from 0% to 98%. Among the successful feasible policies, it catches 0.025 s earlier than balanced cost and 0.047 s earlier than late feasible. The larger gap is about one control tick. Late feasible is more accurate, and balanced cost sits between the two. The report uses time-first as the Task 4 strategy because the task asks for shorter interception time. If the goal were best radial accuracy instead, balanced cost or late feasible would be the better engineering choice. Figure 7 shows this time-accuracy tradeoff directly.

Time-first runs with the tightest motion budget. The NLP often hits the acceleration bound, so it has less room to refine the final pose. Balanced and late-feasible candidates happen later, where the robot has more time to align the hoop with the ball direction. That is why those rows have lower TCP and radial errors. The TCP column is the hoop-center error at the NLP terminal knot. The radial column is the hoop-to-ball error when the true ball crosses the hoop plane. These are different checks. With time-first, the hoop can sweep past the ball, so the crossing instant can be closer than the terminal knot. Late feasible drives TCP error below 2 mm. Its radial error stays near 6 mm because the true ball crosses a few millimetres away from the predicted target.

The open-loop gap is small in the successful time-first catches. The NLP terminal error to the predicted target averaged 13.18 mm. The predicted hoop-crossing radial error averaged 9.36 mm, while the true-ball radial error averaged 11.24 mm. The means differ by 1.87 mm, and the per-seed mean absolute predicted-to-true radial gap is 4.41 mm. This keeps solver error and execution error separate.

Failure Analysis and Interpretation

The only time-first failure is seed 11. Figure 8 shows why it is difficult: this throw is near the 95th percentile of the 50-seed lateral velocity distribution. After the hoop normal was fixed to the TCP z-axis in the Task 3 setup, seed 11's best radial error dropped from 145.2 to 51.5 mm. That is a large improvement, but it still misses the 30 mm tolerance. The same correction recovered seed 41, which now crosses at 24.7 mm.

The seven shortlisted seed 11 candidates all solve the NLP and all cross the hoop plane. None crosses inside the radial tolerance. Their radial errors fall from 758 to 51.5 mm as catch time moves from 1.32 to 1.44 s. The 758 mm case is the earliest candidate, where the UR10 cannot reach the target in time. Every candidate reaches the 2 rad/s^2 acceleration bound. Early candidates saturate joint 3, and later candidates saturate joint 2. This is an acceleration-limited geometry failure, not a failed estimator or a failed NLP solve.

A mitigation run with 16 candidates and a 1.35 m workspace radius did not recover seed 11. The default setting uses 8

candidates and a 1.0 m radius. Recovering this seed would likely require a longer prediction horizon or a different candidate sampler.

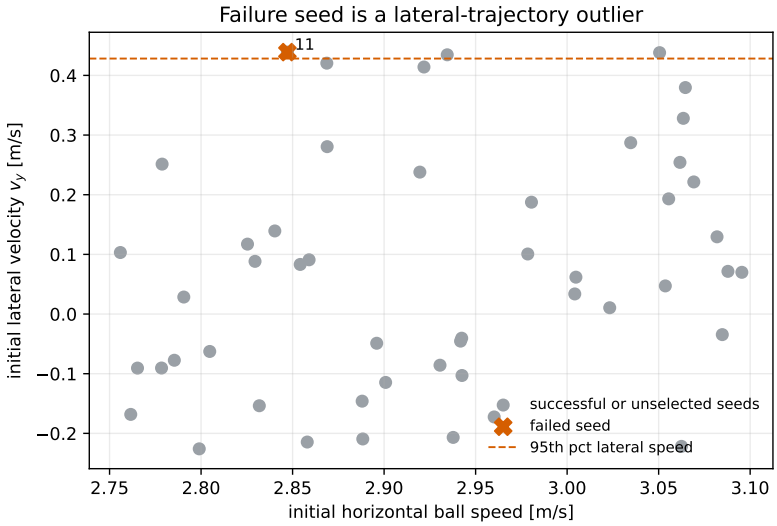

Figure 8: Failure mechanism for seed 11. The red marker is the failed seed and the grey markers are the other seeds; the dashed line marks the empirical 95th percentile of lateral velocity. Seed 11 sits near this high-v_y boundary and fails by acceleration-limited geometry, not estimator divergence or NLP failure.

The final architecture uses direct-transcription optimal control with hard physical constraints and soft secondary objectives. This choice is necessary here. A single-step Jacobian controller can chase a Cartesian target, but it cannot save acceleration for a future catch. A closed-form smooth trajectory can reduce jerk, but it does not enforce hoop crossing, joint limits, and top-side orientation together. The horizon NLP is the smallest formulation in this report that handles all of these requirements at once.

Conclusion

The experiments support the claim made at the start. Prediction is necessary: the reactive controller never reaches the ball within the tolerance. Prediction alone is not enough: the simple geometric selector also gives 0% success. The decisive step is feasibility-aware candidate selection. It keeps the estimator and controller fixed, filters candidates through the UR10 NLP and the true hoop-crossing check, and raises success to 98%. Time-first feasible gives the shortest successful catch among the tested policies. Balanced cost shows what it costs, in time, to buy a few extra millimetres of radial accuracy.

References

[1] Robotics course project description, *A Reverse-Hoops-Playing Robot Arm*, KU Leuven, 2025–2026.

[2] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35–45, 1960. DOI: 10.1115/1.3662552.

[3] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, 106, 25–57, 2006. DOI: 10.1007/s10107-004-0559-y.

[4] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 11, 1–36, 2019. DOI: 10.1007/s12532-018-0139-4.

[5] J. Carpentier et al., "The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," IEEE/SICE International Symposium on System Integration, 2019.